Part 2 Consumer-driven contract testing

At this stage, you must be itching to implement consumer-driven contract testing (CDCT) and get hands-on with all the technical implementation. This part of the book contains the most chapters, diving deep into implementing CDCT for different communication types, the various ways to store your contracts, and how to integrate contract testing as part of your continuous integration/continuous delivery (CI/CD) pipelines.

In chapter 4, you learn how to implement CDCT between a web application consumer and a data provider using the Pact framework in JavaScript. You also learn how to store the contract that the consumer generates into a Pact Broker.

As some of you work with mobile apps, we also have a chapter dedicated to implementing CDCT between an Android mobile app and a data provider. Chapter 5 explores this setup using the Pact framework in Java.

Because CDCT requires both tests to be written on the consumer and provider side, chapter 6 shows how the provider tests need to be set up, with the introduction of provider states. Chapter 7 demonstrates how to introduce contract testing for a GraphQL layer, while chapter 8 shows how to implement contract testing for event-driven architecture.

While we've chosen to use PactFlow as the Pact Broker for this book, there are other options for storing and hosting your contracts, which chapter 9 covers extensively. Finally, chapter 10 demonstrates how to set up contract testing in a CI/CD pipeline using GitHub actions. You learn the basic setup as well as more advanced configurations, such as using webhooks, using the can-i-deploy tool, recording releases or deployments, versioning, and more.